



# Лекция 1. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: [zolotuhin@mirea.ru](mailto:zolotuhin@mirea.ru)



# Условия обучения

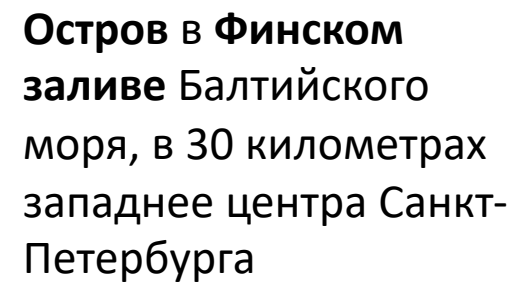
- 8 лекций, 16 практических занятий
- По итогам изучения дисциплины проводится экзамен



# Введение в Kotlin

- **Kotlin (Кóтлин)** - статически типизированный язык программирования, работающий поверх Java Virtual Machine и разрабатываемый компанией JetBrains
- Главными разработчиками Kotlin были Андрей Бреслав, Дмитрий Жемеров и Максим Шаповалов







# История языка

- 2010 – начало разработки внутри JetBrains
- 2011 – представлен общественности
- 2012 – выложен в открытый доступ на GitHub
- 20xx – много разных дополнений
- 15 февраля 2016 – первая версия языка
- 2017 – поддержка Google для разработки под Android
- 2023 – версия языка 1.9



# Почему стоит изучать Kotlin?

- Мульти- и кросс- платформенность
- Совместимость с Java
- Компилируется в байткод JVM или JS
- Улучшенная безопасность
- Улучшенный синтаксис
- Поддержка Google
- Универсальность
- Open Source



# Возможности языка



# Kotlin

Common

JVM

JS

Native



# Статистика. Типы приложений

Что вы разрабатываете на Kotlin?

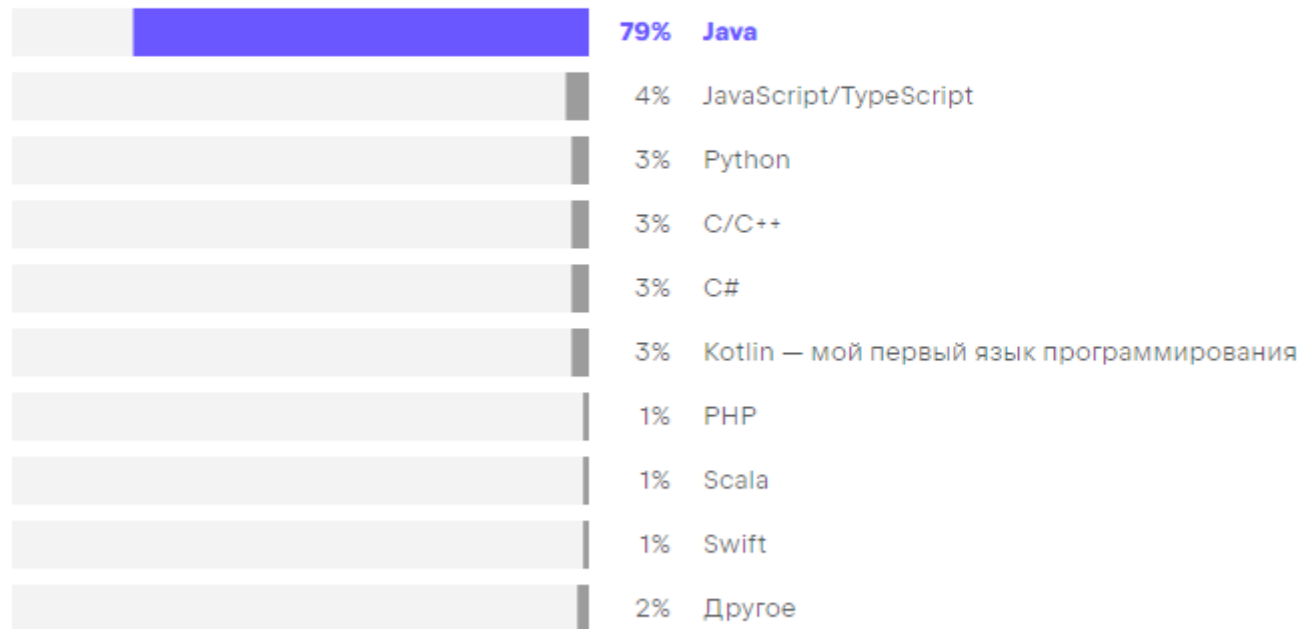






# Статистика. Другие языки

Какой язык программирования был для вас основным, прежде чем вы перешли на Kotlin?



Чаще всего разработчики переходят на Kotlin с Java.



# База Котлина

Java Platform – программная среда, в которой работают приложения, написанные на Java.

Существуют для различных ОС (Windows, Linux, Mac OS).

Включает в себя:

- Java Virtual Machine (JVM) – виртуальная машина Java, программа, интерпретирующая приложения Java;
- Java API – библиотеки программных компонентов (классов и интерфейсов), реализующих стандартный функционал.



# JRE, JDK, JVM

## Java Runtime Environment (JRE)

– минимальная реализация виртуальной машины, необходимая для исполнения Java-приложений, без компилятора и других средств разработки

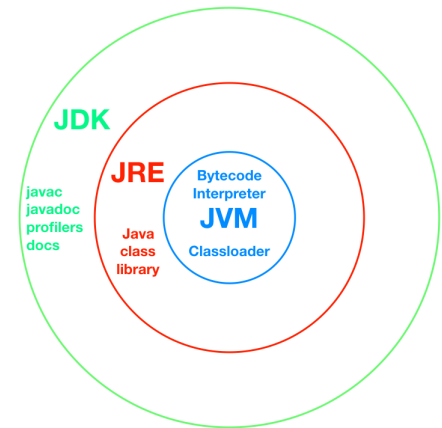
Состоит из JVM и библиотеки классов

## Java Development Kit (JDK) –

комплект разработчика приложений на языке Java, включающий в себя компилятор `javac`, стандартные библиотеки классов Java.

## Java Virtual Machine (JVM) –

виртуальная машина Java исполняет байткод Java, предварительно созданный из исходного текста Java-программы компилятором `javac`.

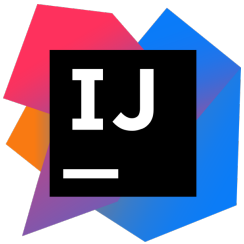




# Среды разработки

**IDE** - Integrated Development Environment

Среды разработки Kotlin



<https://www.jetbrains.com/idea/>



<https://netbeans.apache.org/>



<https://www.eclipse.org/ide/>



<https://code.visualstudio.com/>



# Точка входа в программу

Точкой входа в программу в Kotlin является функция `main()`

```
fun main(){  
    println("Hello Kotlin")  
}
```

Также форма `main` может принимать массив строк `String`

```
fun main(args: Array<String>){  
    println(args.contentToString())  
}
```



# Первая программа

The screenshot shows the IntelliJ IDEA interface. The Project view on the left displays a project named 'untitled' with a 'src' directory. The Main.kt file is open in the editor, containing the following code:

```
1 fun main(){
2     println("Hello World!")
3 }
```

The Run view at the bottom shows the command executed: `C:\Users\svt\jdk\corretto-17.0.8.1\bin\java.exe "-javaagent:C:\Program Files\Hello World!"`. The output is "Hello World!" and the process finished with exit code 0.

The screenshot shows the IntelliJ IDEA interface with the Main.kt file open. The Run view at the bottom shows the same command as the previous screenshot. The output is "Hello World!" and the process finished with exit code 0.

The Project view on the left shows a project named 'untitled' with a 'production' directory. The Main.kt.class file is open in the editor, displaying the decompiled code:

```
1 // IntelliJ API Decompiler stub source generated from a class file, bytecode version: 52.0 (Java 8)
2 // Implementation of methods is not available
3
4 public fun main(): kotlin.Unit { /* compiled code */
5
6 }
```



# Комментарии

/\*

многострочный комментарий

Функция main -

точка входа в программу

\*/

```
fun main(){      // начало функции main
```

```
    println("Hello Kotlin") // вывод строки на консоль
```

```
}                // конец функции main
```



# Переменные, ключевое слово `val` (`value`)

***Общая форма записи переменных:***

***[ `val` | `var` имя\_переменной: тип\_переменной ]***

**`val` – неизменяемые переменные**

**`val a: Int = 1` // Инициализация при объявлении**

**`val b = 1` // Тип `Int` определен автоматически**

**`val c: Int` // Указывать тип обязательно, если переменная не инициализирована сразу**

**`c = 1` // Последующее присвоение**





# Переменные, ключевое слово `var` (variable)

Изменяемые переменные объявляются с помощью ключевого слова **`var`**

**`var x = 5` // Тип `Int` определен автоматически**

**`x += 1`**

```
fun main() {  
    var a: Int  
    a = 7  
    println(a) // 7  
    a = 18  
    println(a) // 18  
}
```



# Типы данных

## Целочисленные типы:

- **Byte:** хранит целое число от -128 до 127 и занимает 1 байт
- **Short:** хранит целое число от -32 768 до 32 767 и занимает 2 байта
- **Int:** хранит целое число от -2 147 483 648 (-2<sup>31</sup>) до 2 147 483 647 (2<sup>31</sup> - 1) и занимает 4 байта
- **Long:** хранит целое число от -9 223 372 036 854 775 808 (-2<sup>63</sup>) до 9 223 372 036 854 775 807 (2<sup>63</sup>-1) и занимает 8 байт

**val** threeBillion = 3000000000 *// Long*

**val** oneLong = 1L *// Long*



# Типы данных

Также добавлена поддержка для целочисленных типов без знака:

- **UByte**: хранит целое число от 0 до 255 и занимает 1 байт
- **UShort**: хранит целое число от 0 до 65 535 и занимает 2 байта
- **UInt**: хранит целое число от 0 до  $2^{32} - 1$  и занимает 4 байта
- **ULong**: хранит целое число от 0 до  $2^{64} - 1$  и занимает 8 байт



# Типы данных

## Числа с плавающей точкой:

- **Float:** хранит число с плавающей точкой от  $-3.4 \cdot 10^{38}$  до  $3.4 \cdot 10^{38}$  и занимает 4 байта

Чтобы явно указать тип Float, добавьте после значения `f` или `F`. **Если такое значение содержит более 6-7 разрядов, оно будет округлено.**

`val eFloat = 2.7182818284f` // *Float, фактическое значение 2.7182817*

- **Double:** хранит число с плавающей точкой от  $\pm 5.0 \cdot 10^{-324}$  до  $\pm 1.7 \cdot 10^{308}$  и занимает 8 байт.

`val e = 2.7182818284` // *Double*



# Типы данных. Символы, строки

Символьные данные представлены типом **Char**. Он представляет отдельный символ, который заключается в одинарные кавычки.

Обратите внимание, что символы (characters) в языке Kotlin не являются числами (в отличие от Java).

```
val a: Char = 'A'
```

```
val b: Char = 'B'
```

```
val c: Char = 'C'
```

```
val name: String = "Peter"
```



# Преобразования типов

Из-за разницы в представлениях, меньшие типы *не являются подтипами* БОльших типов.

Как следствие, неявное преобразование меньших типов в БОльшие НЕ происходит. Это значит, что мы не можем присвоить значение типа Byte переменной типа Int без явного преобразования.

**val b: Byte = 1 // всё хорошо, литералы проверяются статически**

**val i: Int = b // ОШИБКА**

**val i1: Int = b.toInt()**



# Преобразования типов

Каждый численный тип поддерживает следующие преобразования:

- `toByte(): Byte`
- `toShort(): Short`
- `toInt(): Int`
- `toLong(): Long`
- `toFloat(): Float`
- `toDouble(): Double`
- `toChar(): Char`

Часто необходимости в явных преобразованиях нет, поскольку тип выводится из контекста, а арифметические действия перегружаются для подходящих преобразований.

**val** `l = 1L + 3 // Long + Int => Long`



# КОНСОЛЬНЫЙ ВВОД/ВЫВОД

Вывод информации на консоль:

**print()** – выводит свой аргумент в стандартный поток вывода

```
print("Hello ")
```

```
print("world!")
```

**println()** – выводит аргумент и добавляет перевод строки





# КОНСОЛЬНЫЙ ВВОД/ВЫВОД

Стандартный метод считывания – функция **readLine()**.

Минусы:

- Функция возвращает строку или null,
- Для возвращения числа нужно приведение типов,
- При считывании нескольких чисел подряд возникают сложности.

```
print("Введите имя: ") // Петя
```

```
val name = readLine()
```

```
println("Ваше имя: $name") // Ваше имя: Петя
```



# КОНСОЛЬНЫЙ ВВОД/ВЫВОД

Следующий метод считывания – нестандартная функция **readln()**.

Минусы:

- Функция возвращает строку,
- Для возвращения числа нужно приведение типов,
- Выбрасывает исключение EOF has already been reached.

```
print("Введите имя: ") // Петя
```

```
val name = readln()
```

```
println("Ваше имя: $name") // Ваше имя: Петя
```



# КОНСОЛЬНЫЙ ВВОД/ВЫВОД

Еще один способ – класс Scanner из Java.

```
val scan = java.util.Scanner(System.`in`)
```

```
val a = scan.nextInt() // считывание числа в переменную a
```

Другие методы класса Scanner:

**next():** считывает введенную строку до первого пробела

**nextLine():** считывает всю введенную строку

**nextInt():** считывает введенное число int

**nextDouble():** считывает введенное число double

**nextFloat():** считывает введенное число float

...



# Арифметические операции

Kotlin поддерживает базовые арифметические операции:

**+** (**сложение**): возвращает сумму двух чисел.

**-** (**вычитание**): возвращает разность двух чисел.

**\*** (**умножение**): возвращает произведение двух чисел.

**/** (**деление**): возвращает частное двух чисел.

**%**: возвращает остаток от целочисленного деления двух чисел.

**++** (**инкремент**): увеличивает значение на единицу.

**--** (**декремент**): уменьшает значение на единицу.



# Арифметические операции

Также, есть ряд операций присвоения, которые сочетают арифметические операции и присвоение:

$+=$  : присваивание после сложения

$-=$  : присваивание после вычитания

$*=$  : присваивание после умножения

$/=$  : присваивание после деления

$\%=$  : присваивание после деления по модулю



# Поразрядные операции

- **shl**: сдвиг битов числа со знаком влево
- **shr**: сдвиг битов числа со знаком вправо
- **ushr**: сдвиг битов беззнакового числа вправо
- **and**: побитовая операция AND (конъюнкция)
- **or**: побитовая операция OR (дизъюнкция)
- **xor**: побитовая операция XOR
- **inv**: логическое отрицание или инверсия



# Условные выражения

> (больше чем): возвращает true, если первый операнд больше второго. Иначе возвращает false

< (меньше чем): возвращает true, если первый операнд меньше второго. Иначе возвращает false

>= (больше чем или равно): возвращает true, если первый операнд больше или равен второму

<= (меньше чем или равно): возвращает true, если первый операнд меньше или равен второму.

== (равно): возвращает true, если оба операнда равны. Иначе возвращает false

!= (не равно): возвращает true, если оба операнда НЕ равны



# Логические операции

- **and**: возвращает true, если оба операнда равны true.
- **or**: возвращает true, если хотя бы один из операндов равен true.
- **xor**: возвращает true, если только один из операндов равен true. Если операнды равны, возвращается false
- **!**: возвращает true, если операнд равен false. И, наоборот, если операнд равен true, возвращается false.

```
val a = true
```

```
val b = !a // false
```

```
val c = !b // true
```

```
val a = true
```

```
val b = a.not() // false
```

```
val c = b.not() // true
```





# Диапазоны (интервалы)

Диапазон представляет набор значений или некоторый интервал. Для создания диапазона применяется оператор ...:

**val range = 1..5 // диапазон [1, 2, 3, 4, 5]**

**val range = "a".."d" // диапазон [a, b, c, d]**

С помощью функции `downTo` можно построить диапазон в обратном порядке:

**val range2 = 5 downTo 1 // диапазон [5, 4, 3, 2, 1]**



# Диапазоны (интервалы)

Функция **step** позволяет задать шаг, на который будут изменяться последующие элементы:

```
val r1 = 1..10 step 2      // диапазон [1, 3, 5, 7, 9]
```

```
val r2 = 10 downTo 1 step 3 // диапазон [10, 7, 4, 1]
```

Функция **until** позволяет не включать верхнюю границу в диапазон:

```
val r3 = 1 until 9        // диапазон [1, 2, 3, 4, 5, 6, 7, 8]
```

```
val r4 = 1 until 9 step 2  // диапазон [1, 3, 5, 7]
```



# Диапазоны (интервалы)

**in:** возвращает true, если операнд имеется в некоторой последовательности.

```
val a = 7
```

```
val b = a in 1..9    // true
```

```
val c = 5
```

```
val d = c in 11..14  // false
```

```
val a = 4
```

```
val b = a !in 1..3 // true
```



# Условные конструкции

```
val a = 5
```

```
if (a == 5){  
    println("a равно 5")  
}
```

```
val a = 5
```

```
if (a == 5){  
    println("a равно 5")  
}  
else {  
    println("a не равно 10")  
}
```

```
val a = 5
```

```
if (a == 5)  
    println("a равно 5")  
  
else  
    println("a не равно 10")
```

```
val a = 5
```

```
if (a == 5){  
    println("a равно 5")  
}  
else if (a == 4){  
    println("a равно 4")  
}  
else if (a == 3){  
    println("a равно 3")  
}  
else {  
    println("a не определено")  
}
```



# Условные конструкции

Конструкция if может возвращать значение. Например:

```
val a = 10
```

```
val b = 20
```

```
val c = if (a > b) a else b
```

```
println(c) // 20
```



# Условные конструкции

```
val max = if (a > b) {  
    print("возвращаем a")  
    a  
}  
else {  
    print("возвращаем b")  
    b  
}
```

При реализации конструкции if в качестве выражения – использование ветки else является обязательным



# Конструкция when

**Формальная запись:**

*when(объект){*

*значение1 -> действия1*

*значение2 -> действия2*

*значениеN -> действияN*

*}*



# Конструкция when

```
when (x) {  
  1 -> print("x == 1")  
  2 -> print("x == 2")  
  else -> { // обратите внимание на блок  
    print("x is neither 1 nor 2")  
  }  
}
```





# Конструкция when

```
when (x) {  
    0, 1 -> print("x == 0 or x == 1")  
    else -> print("otherwise")  
}
```

Помимо констант в ветках можно использовать произвольные выражения:

```
when (x) {  
    parseInt(s) -> print("s encodes x")  
    else -> print("s does not encode x")  
}
```



# Конструкция when

```
val a = 5
```

```
when(a){
```

```
    in 5..10 -> println("а в диапазоне от 5 до 10")
```

```
    in 15..21 -> println("а в диапазоне от 15 до 21")
```

```
    !in 10..20 -> println("а вне диапазона от 10 до 20")
```

```
    else -> println("неопределенное значение")
```

```
}
```



# Конструкция when

```
val sum = 5555
```

```
val rate = when(sum){  
    in 100..999 -> 5  
    in 1000..9999 -> 10  
    else -> 15  
}  
println(rate)    // 10
```

Также как и с if, при реализации конструкции when в качестве выражения – использование ветки else является обязательным



# Циклы, for

## Формальная запись

```
for (переменная in последовательность){  
    выполняемые инструкции  
}
```

```
for (n in 1..9){  
    print("${n * n} \t")  
}
```

```
// Вывод: 1  4  9  16  25  36  49  64  81
```



# Циклы, while, do..while

```
var i = 10
while(i > 0){
    println(i*i)
    i--;
}
```

```
var i = -1
do{
    println(i*i)
    i--;
}
while(i > 0)
```



# Операторы continue и break

```
for(n in 1..8){  
    if(n == 5)  
        continue;  
    println(n * n)  
}
```

```
for(n in 1..5){  
    if(n == 5) break;  
    println(n * n)  
}
```



# Строковые литералы

Итерация по строке

```
for (c in str) {  
    println(c)  
}
```

Строковые литералы: **val s = "Hello, world!\n"**

Многострочная запись

```
val text = """  
    for (c in "foo")  
        print(c)  
    """
```



# Строковые шаблоны

Строки могут содержать шаблонные выражения, т.е. участки кода, которые выполняются, а полученный результат встраивается в строку. Шаблон начинается со знака доллара (\$) и состоит либо из простого имени (например, переменной):

```
val i = 10
```

```
val s = "i = $i" // evaluates to "i = 10"
```

либо

```
val s = "abc"
```

```
val str = "$s.length is ${s.length}" // evaluates to  
"abc.length is 3"
```





# Массивы

```
val numbers: Array<Int>
```

С помощью встроенной функции `arrayOf()` можно передать набор значений

```
val numbers: Array<Int> = arrayOf(1, 2, 3, 4, 5)
```

```
val n = numbers[1] // получаем второй элемент n=2
```

```
numbers[2] = 7    // переустанавливаем третий элемент
```

```
println("numbers[2] = ${numbers[2]}") // numbers[2] = 7
```



# Массивы

Также инициализировать массив значениями можно следующим способом:

```
val numbers = Array(3, {5}) // [5, 5, 5]
```

```
var i = 1;
```

```
val numbers = Array(3, { i++ * 2 }) // [2, 4, 6]
```



# Перебор массива

Для перебора массивов можно применять цикл for:

```
val numbers = arrayOf(1, 2, 3, 4, 5)
```

```
    for(number in numbers){
```

```
        print("$number \t")
```

```
    }
```

```
// 1 2 3 4 5
```



# Перебор массива

Также можно перебирать массивы и других типов:

```
val people = arrayOf("Nick", "Peter", "Tom")  
  for(person in people){  
    print("$person \t")  
  }  
// Nick   Peter   Tom
```



# Перебор массива

Другой тип цикла для перебора:

```
val people = arrayOf("Tom", "Sam", "Bob")
```

```
var i = 0
```

```
while(i in people.indices){
```

```
    print(people[i] + "\t")
```

```
    i++;
```

```
}
```

```
// Tom Sam Bob
```

```
// Выражение i in people.indices проверяет, находится  
ли значение i внутри диапазона индексов элементов  
массива people
```



# Массивы базовых типов

Для упрощения создания массива в Kotlin определены дополнительные типы **BooleanArray**, **ByteArray**, **ShortArray**, **IntArray**, **LongArray**, **CharArray**, **FloatArray** и **DoubleArray**, которые позволяют создавать массивы для определенных типов. Например, тип **IntArray** позволяет определить массив объектов **Int**, а **DoubleArray** - массив объектов **Double**

```
val numbers: IntArray = intArrayOf() // можно оставить  
неинициализированным
```

```
val numbers: IntArray = intArrayOf(1, 2, 3, 4, 5)
```

```
val chars = charArrayOf('a', 'b', 'c')
```



# Работа с массивами

```
val arrayOfIngredients: Array<String> = arrayOf("яйцо",  
"масло", "соль", "зелень", "помидор")
```

```
print(arrayOfIngredients.size) // 5
```

```
print(arrayOfIngredients.indexOf("зелень")) // элемент  
с индексом 3
```

Чтение и запись по индексу с помощью функций `set()` и `get()`

```
arrayOfIngredients.set(4, "бекон") // четвертый  
элемент теперь бекон
```

```
println(arrayOfIngredients.get(4)) // бекон
```



# Двумерные массивы

Определение двумерного массива:

```
val table: Array<Array<Int>> = Array(3, { Array(3, {0}) })
```

Данный массив будет иметь 3 элемента – три строки, каждая из которых будет иметь по 3 элемента, каждая из которых равна 0.

Изменим значения массива:

```
table[0] = arrayOf(1, 2, 3)
```

```
table[1] = arrayOf(4, 5, 6)
```

```
table[2] = arrayOf(7, 8, 9)
```





# Двумерные массивы

Вывод массива на экран:

```
for (row in table){  
    for (cell in row){  
        print("$cell \t")  
    }  
    println()  
}  
  
// 1  2  3  
   4  5  6  
   7  8  9
```



Спасибо за внимание!